Tutorial 1: Building a Perceptron

Name: Laiba Niazi

Roll Number: 536672

1. Introduction

The objective of this tutorial was to implement a linear binary classifier using the **Perceptron** model from the scikit-learn library. The study focuses on distinguishing the **Iris-setosa** species from other species in the dataset based on morphological features.

2. Dataset Analysis

The analysis utilizes the **Iris dataset** sourced from the UCI Machine Learning Repository.

- **Features**: The dataset includes four numeric attributes: sepal length, sepal width, petal length, and petal width.
- **Target**: The original dataset contains three classes: Iris-setosa, Iris-versicolor, and Iris-virginica.
- **Preprocessing**: The shuffle utility was used to randomize the row order.

```
df = shuffle(df)
df.head()
```

[4] Python

	# 5.1	# 3.5	# 1.4	# 0.2	Iris-setosa
23	4.8	3.4	1.9	0.2	Iris-setosa
88	5.5	2.5	4.0	1.3	Iris-versicolor
131	6.4	2.8	5.6	2.2	Iris-virginica
122	6.3	2.7	4.9	1.8	Iris-virginica
94	5.7	3.0	4.2	1.2	Iris-versicolor

3. Methodology

The core of the tutorial involved configuring and training the Perceptron model.

Model Configuration

The Perceptron was initialized with a learning rate (eta0) of **0.1** and a maximum iteration count of **10**.

Task Breakdown

- **Label Encoding**: The multi-class problem was converted into a binary classification problem. Initially, Iris-setosa was labeled as **1** and all others as **-1**.

```python
train_data, test_data, train_labels, test_labels = train_test_split(X,y,test_size=0.25)

train_labels = np.where(train_labels == 'Iris-setosa', 1, -1)
test_labels = np.where(test_labels == 'Iris-setosa', 1, -1)

print("Train data: ", train_data[0:2])
print("Train labels: ",train_labels[0:2])
print("Test data: ",test_data[0:2])
print("Test labels: ",test_labels[0:2])
```

[8] Python

```
Train data:  [[7.  3.2 4.7 1.4]
 [6.8 3.2 5.9 2.3]]
Train labels:  [-1 -1]
Test data:  [[5.5 2.3 4.  1.3]
 [7.3 2.9 6.3 1.8]]
Test labels:  [-1 -1]
```

- **Task 1 (Manual Input)**: A system was implemented where a user can provide flower dimensions (sepal_length, etc.) for real-time prediction.

```python
print("Enter manual input for flower features:")
sepal_length = float(input("Sepal Length: "))
sepal_width = float(input("Sepal Width: "))
petal_length = float(input("Petal Length: "))
petal_width = float(input("Petal Width: "))


manual_input = np.array([[sepal_length, sepal_width, petal_length, petal_width]])
print("Manual Input: ", manual_input)
```

[15] Python

```
Enter manual input for flower features:
Manual Input:  [[4.3 5.6 4.4 2.6]]
```

```python
y_manual_pred = perceptron.predict(manual_input)
print('Prediction for manual input: ', y_manual_pred)
```

[17] Python

```
Prediction for manual input:  [-1]
```

4. Results

The model demonstrated high efficiency in both test runs:

- **Accuracy**: In both the -1/1 and 0/1 encoding scenarios, the model achieved an **accuracy of 100%** on the test data.

```python
y_preds = perceptron.predict(test_data)
accuracy = accuracy_score(test_labels, y_preds)
print('Accuracy: ', round(accuracy , 2) * 100, "%")
```

[24] Python

```
Accuracy:  100.0 %
```

5. Conclusion

The Perceptron proves to be an effective linear classifier for the Iris dataset. Because the **Iris-setosa** class is linearly separable from the other species, the model reached perfect accuracy within only 10 iterations. This tutorial successfully demonstrates the fundamental pipeline of data loading, preprocessing, model training, and evaluation.